

18

Moving a Model to Production

Moving a model to production is a step toward enabling the consumption of our model by an external party. We should expose our model to the world and start rendering predictions on real, unseen input.

It is not sufficient to have a trained PyTorch model for deployment. We need additional server components to set up communication channels from the real world to the PyTorch model and back to the real world. It is important that we know how to create an API (through which a user can interact with the model), wrap it as a self-contained application (so that it can be deployed on any computer), and ship it to the cloud – so that anybody with the required URL and credentials can interact with the model. To successfully move a model to production, all these steps are necessary.

In addition, we will have to deal with constraints around the latency of predictions (the time taken to get a prediction) and the size of the model (when deploying on edge devices like mobiles/watches) without sacrificing accuracy.

Furthermore, as part of production support, we will have to keep a tab on the inputs to the model once it is deployed, and then check if there is a considerable drift (deviation) between the images that were used to train the model and the images that are fed to it during real-world deployment.

In this chapter, we will deploy a simple application and build a mechanism to identify input drift.

The following topics will be covered in this chapter:

- Understanding the basics of an API
- Creating an API and making predictions on a local server
- Dockerizing and deploying the model on cloud
- Identifying data drift
- Building a vector store using **Facebook AI Similarity Search (FAISS)**

All code snippets within this chapter are available in the **Chapter18** folder of the Github repository at <https://bit.ly/mcwp-2e>.

As a bonus, we also cover the following topic in our GitHub repository:



- Converting a model into the **Open Neural Network Exchange (ONNX)** format

As the field evolves, we will periodically add valuable supplements to the GitHub repository. Do check the

`supplementary_sections` folder within each chapter's directory for new and useful content.

It is important to note the typical workflow to deploy a model, which is as follows:

1. Create an API and make predictions on the local server
2. Containerize the application
3. Deploy the Docker container on the cloud
4. Build a vector store of training images
5. Identify if there is a deviation (drift) in real-world images (so that we know if we need to fine-tune a model)

Understanding the basics of an API

By now, we know how to create a deep learning model for various tasks. It accepts/returns tensors as input/output. But an outsider such as a client/end user would talk only in terms of images and classes.

Furthermore, they would expect to send and receive input/output over channels that might have nothing to do with Python. The internet is the easiest channel to communicate on. Hence, for a client, the best-case deployment scenario would be if we could set up a publicly available URL and ask them to upload their images there. One such paradigm is called an **Application Programming Interface (API)**, which has standard protocols that accept input and post output over the internet while abstracting the user away from how the input is processed or the output is generated.

Some common protocols in APIs are `POST`, `GET`, `PUT`, and `DELETE`, which are sent as **requests** by the client to the host server along with relevant data. Based on the request and data, the server performs the relevant task and returns appropriate data in the form of a **response**, which the client can use in their downstream tasks. In our case, the client will send a POST request with an image as a file attachment. We should save the file, process it, and return the appropriate class as a response to the request, and our job is done.

Requests are organized data packets sent over the internet to communicate with API servers. Typically, the components in a request are as follows:

- **An endpoint URL:** This would be the address of the API service. For example, <https://www.packtpub.com/> would be an endpoint to connect to the Packt Publishing service and browse through the catalog of their latest books.
- **A collection of headers:** This information helps the API server return output; for instance, if the header contains information that the client is on a mobile, then the API can return an HTML page with a layout that is mobile-friendly.
- **A collection of queries:** This ensures that only related items from the server database are fetched. For example, a search string of `PyTorch` will return only PyTorch-related books in the previous example. Note

that, in this chapter, we will not work on queries, as a prediction on images does not require querying – it requires a filename.

- **A list of files:** This could be uploaded to the server or, in our case, used to make deep learning predictions.

cURL is a computer software project that provides a library and command-line tool to transfer data using various network protocols. It is one of the most lightweight, commonly used, and simple applications to call API requests and get back responses.

To illustrate this, we will implement a readily available Python module called FastAPI in the following sections that will enable us to do the following:

1. Set up a communication URL.
2. Accept input from a wide variety of environments/formats when it is sent to the URL.
3. Convert every form of input into the exact format that the machine learning model needs as input.
4. Make predictions with the trained deep learning-based model.
5. Convert predictions into the right format and respond to the client's request with the prediction.

We will use the **Surface-Defect Dataset (SDD)**

(<https://www.vicos.si/resources/kolektorsdd/>) as an example to demonstrate these concepts.

After understanding the basic setup and code, you can create APIs for any kind of deep learning task and serve predictions through a URL on your local machine. The next logical step is to containerize the application and deploy it on the cloud so that the application is accessible from anywhere in the world. The deployed application will then need support as it is important for us to understand the methods to identify deviations in the real-world data if the model is misbehaving.

In the upcoming sections, we will cover how to wrap an application in a self-contained Docker image that can be shipped and deployed anywhere on the cloud. Let's use FastAPI, a Python library, to create the API and verify that we can make predictions directly from the terminal (without Jupyter notebooks).

Creating an API and making predictions on a local server

In this section, we will learn about making predictions on a local server (that has nothing to do with the cloud). At a high level, this involves the following steps:

1. Installing FastAPI
2. Creating a route to accept incoming requests
3. Saving an incoming request on disk
4. Loading the requested image, and then preprocessing and predicting with the trained model

5. Postprocessing the results and sending back the predictions as a response to the same incoming request



All of the steps in this section are summarized as a video walk-through here: <https://tinyurl.com/MCVP-Model2FastAPI>.

Let's begin by installing FastAPI.

Installing the API module and dependencies

Since FastAPI is a Python module, we can use `pip` for installation and get ready to code an API. We will now open a new terminal and run the following command:

```
$pip install fastapi uvicorn aiofiles jinja2
```

We have installed a couple more dependencies that are needed with FastAPI. `uvicorn` is a minimal low-level server/application interface for setting up APIs. `aiofiles` enables the server to work asynchronously with requests, such as accepting and responding to multiple independent parallel requests at the same time. These two modules are dependencies for FastAPI, and we will not directly interact with them.

Let's create the required files and code them in the next section.

Serving an image classifier

In this section, we'll learn about deploying a model locally so that we can receive predictions from an endpoint.

The first step is to set up a folder structure, as shown here:

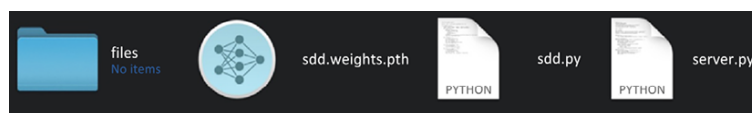


Figure 18.1: Folder structure for model serving

The setup is quite minimal, as shown here:

- The `files` folder is going to act as the download location for incoming requests.
- `sdd.weights.pth` contains the weights of our trained SDD model.
- `sdd.py` will contain logic to load the weights, accept incoming images, and preprocess, predict, and postprocess the predictions.
- `server.py` will contain FastAPI functionalities that can set up a URL, accept client requests from the URL, send/receive input/output from `sdd.py`, and send the output as responses to the client requests.

Note



The `files` folder is empty and is only used to store uploaded files.

We are assuming we have the weights of the trained model as `sdd.weights.pth`.

The training process is similar to how we trained for image classification in *Chapter 4*.

The notebook associated with training the model is provided in the first section of the `vector_stores.ipynb` notebook, in the `Chapter18` folder of the book's GitHub repository.

Let's understand what `sdd.py` and `server.py` constitute and code them now.

sdd.py

As discussed, the `sdd.py` file should have the logic to load the model and return predictions of a given image.

We are already familiar with how to create a PyTorch model. The only additional component to the class is the `predict` method, which is there for doing any necessary preprocessing on the image and postprocessing on the results. You can follow these steps:



The following code is available as `sdd.py` in the `Chapter18` folder of the book's GitHub repository at <https://bit.ly/mcnp-2e>.

1. We first create the `model` class that constitutes the architecture of the model:

```
from torch_snippets import *
class SDD(nn.Module):
    classes = ['defect', 'non_defect']
    def __init__(self, model, device='cpu'):
        super().__init__()
        self.model = model.to(device)
        self.device = device
```

2. The following code block highlights the forward method:

```
@torch.no_grad()
def forward(self, x):
    x = x.view(-1, 3, 224, 224).to(device)
    pred = self.model(x)
    conf = pred[0][0]
    cls = np.where(conf.item() < 0.5, 'non_defect', 'defect')
    print(cls)
    return cls.item()
```

In the preceding code, we first reshape the input image, pass it through the model, and fetch the predicted class corresponding to the image.

The following code block highlights the `predict` method to do the necessary preprocessing and postprocessing:

```
def predict(self, image):
    im = (image[:,:,:-1])
    im = cv2.resize(im, (224,224))
    im = torch.tensor(im/255)
    im = im.permute(2,0,1).float()
    cls = self.forward(im)
    return {"class": cls}
```

In summary, in the preceding steps, in the `__init__` method, we initialize the model (where we have loaded the pretrained weights in the previous step). In the `forward` method, we pass an image through the model and fetch predictions. In the `predict` method, we load an image from a predefined path, preprocess the image before passing it through the `forward` method of the model, and wrap the output in a dictionary while returning the predicted class.

server.py

This is the portion of code in the API that connects the user's request with the PyTorch model. Let's create the file step by step:



The following code is available as `server.py` in the Chapter18 folder of the book's GitHub repository at <https://bit.ly/mcnp-2e>.

1. Load the libraries:

```
import os, io
from sdd import SDD
from PIL import Image
from fastapi import FastAPI, Request, File, UploadFile
```

`FastAPI` is the base server class that will be used to create an API.

`Request`, `File`, and `UploadFile` are proxy placeholders for a client request and the files they will upload. For more details, you are encouraged to go through the official FastAPI documentation here:

<https://fastapi.tiangolo.com/>.

2. Load the model:

```
# Load the model from sdd.py
device = 'cuda' if torch.cuda.is_available() else 'cpu'
model = SDD(torch.load('sdd.weights.pth', map_location=device))
```

3. Create an `app` model that can supply us with a URL for uploading and displaying:

```
app = FastAPI()
```

4. Create a URL at `"/predict"` so that the client can send `POST` requests to `"<hosturl>/predict"` (we will learn about `<hosturl>`, which is the server, in the next section) and receive responses:

```
@app.post("/predict")
def predict(request: Request, file: UploadFile=File(...)):
    content = file.file.read()
    image = Image.open(io.BytesIO(content)).convert('L')
    output = model.predict(image)
    return output
```

That's it! We have all the components to leverage our image classifier to make predictions over our local server. Let's set up the server and make some predictions over the local server.

Running the server

Now that we have set all the components up, we are ready to run the server. Open a new terminal and `cd` the folder that contains `sdd.py`, `server.py`:

1. Run the server:

```
$ uvicorn server:app
```

You will see a message like so:

```
INFO: Started server process [71575]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: 127.0.0.1:62268 - "GET / HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:62268 - "GET /favicon.ico HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:62274 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:62274 - "GET /openapi.json HTTP/1.1" 200 OK
```

Figure 18.2: Messages during Application startup

The `Uvicorn running on ...` message indicates that the server is up and running.

2. To fetch predictions, we will run the following in a separate, new terminal to fetch predictions for a sample image present in `/home/me/Pictures/defect.png`:

```
$ curl -X POST "http://127.0.0.1:8000/predict" -H "accept: application/json" -H "Content-Type:
```

The major components of the preceding line of code are as follows:

- **REST API method:** The method used is `POST`, which indicates that we want to send our own data to the server.
- **URL – server address:** The server host URL is `http://127.0.0.1:8000/` (which is the local server, with `8000` as the

default port) and `/predict/` is the route given to the client to create POST requests; future clients must upload their data to the URL `http://127.0.0.1:8000/predict`.

- **Headers:** The request has components in the form of `-H` flags. These explain additional information, such as the following:
 1. What the input content type is going to be – `multipart/form-data` – which is API jargon for saying the input data is in the form of a file.
 2. What the expected output type is – `application/json` – which means the JSON format. There are other formats, such as XML, text, and `octet-stream`, which are applicable based on the complexity of the output being generated.
- **Files:** The final `-F` flag is pointing to the location where the file that we want to upload exists, and what its type is.

The output dictionary, once we run the preceding code, will be printed in the terminal.

We can now fetch model predictions from our local server.

Now that we have learned about building a server, in the next section, we will learn about containerizing the application so that it can be run from any system.

Containerizing the application

We would rather install one package that has everything than install multiple individual packages (such as the individual modules and code required to run the application) and connect them later. Thus, it becomes important that we can wrap the entire code base and modules into a single package (something like a `.exe` file in Windows) so that the package can be deployed with as little as one command, still ensuring that it works the same on all hardware. To this end, we need to learn how to work with Docker, which is essentially a condensed operating system with code. The created Docker containers are lightweight and will perform only the tasks that we want them to perform. In our example, the Docker image we will create will run the API for the task of predicting the class of SDD images. But first, let's understand some Docker jargon.

A **Docker image** is a standard unit of software that packages up code and all its dependencies. This way, the application runs quickly and reliably from one computing environment to another. A Docker image is a lightweight, standalone, executable package of software that includes everything needed to run an application: code, runtime, system tools, system libraries, and settings.

A **Docker container** is a snapshot of the image that will be instantiated wherever it needs to be deployed. We can create any number of Docker containers from a single image, and they will perform the same task. Think of an image as the blueprint and a container as the instances created from that blueprint.

At a high level, we will learn how to perform the following tasks:

1. Create a Docker image.
2. Create a Docker container out of it and test it.

Let's start by creating a Docker container.

Building a Docker image

In the previous section, we built an API that takes an image and returns the class of the image and the probability associated with that class of image on a local server. Now, it's time to wrap our API in a package that can be shipped and deployed anywhere.



Ensure Docker is installed on your machine. You can refer to <https://docs.docker.com/get-docker/> for instructions on the installation.

There are three steps to the process of creating a Docker container:

1. Create a `requirements.txt` file.
2. Create a Dockerfile.
3. Build a Docker image and create a Docker container.



The code in the following sections is also summarized as a video walk-through here: <https://tinyurl.com/MCVP-2E-model2fastapi-docker>.

We will go through and understand these four steps now, and in the next section, we will learn how to ship the image to AWS servers.

Creating the requirements.txt file

We need to tell the Docker image which Python modules to install to run the application. The `requirements.txt` file contains a list of all these Python modules:

1. Open a terminal and go to the folder that contains `sdd.py`, `server.py`. Next, we will create a blank virtual environment and activate it in our local terminal in the `root` folder:

```
$ python3 -m venv fastapi-venv
$ source fastapi-env/bin/activate
```

The reason why we create a blank virtual environment is to ensure that *only* the required modules are installed in the environment so that, when shipping, we don't waste valuable space.

2. Install the required packages (`fastapi`, `uvicorn`, `aiofiles`, `torch`, and `torch_snippets`) to run the SDD app:

```
$ pip install fastapi uvicorn aiofiles torch torch_snippets
```

3. In the same terminal, run the following command to install all the required Python modules:

```
$ pip freeze > requirements.txt
```

The preceding code fetches all the Python modules and their corresponding version numbers into the `requirements.txt` file, which will be used to install dependencies in the Docker image.

Now that we have all the prerequisites, let's create the Dockerfile in the next section.

Creating a Dockerfile

As introduced in the preceding section, the Docker image is a self-contained application, complete with its own operating system and dependencies. Given a computation platform (such as an EC2 instance), the image can act independently and perform the tasks that it is designed to perform. For this, we need to provide a Docker application with the necessary instructions – dependencies, code, and commands – to launch applications.

Let's create these instructions in a text file called `Dockerfile` in the `root` directory of our SDD project that contains `server.py`, `sdd.py` (which we already placed after creating the project folder). The file needs to be named `Dockerfile` (no extension) as a convention. The content of the text file follows:



The following code is available in `Dockerfile` within the `Chapter18` folder of the book's GitHub repository at <https://bit.ly/mcwp-2e>.

```
FROM tiangolo/uvicorn-gunicorn-fastapi:python3.7
COPY ./requirements.txt /app/requirements.txt
RUN pip install --no-cache-dir -r requirements.txt
WORKDIR /app
COPY . /app
EXPOSE 5000
CMD ["uvicorn", "server:app", "--host", "0.0.0.0"]
```

Let's understand the preceding code step by step:

1. `FROM` instructs us which operating system base to use. The `tiangolo/uvicorn-gunicorn-fastapi:python3.7` location is an address that is parsed by Docker from the internet, and it fetches a base image that has already installed Python and other FastAPI modules.
2. Next, we copy the `requirements.txt` file that we created. This provides the packages that we want to install. In the next line, we ask the image to install packages using `pip install`.
3. `WORKDIR` is the folder where our application will run. Hence, we create a new folder named `/app` in the Docker image and copy the contents of the `root` folder into the `/app` folder of the image.

4. Finally, we run the server as we did in the previous section.

This way, we have set up a blueprint to create a completely new operating system and filesystem (think of it as a new Windows installable CD) from scratch, which is going to contain only the code that we specify and run only one application, which is FastAPI.

Building a Docker image and creating a Docker container

Note that, so far, we have only created a blueprint for the Docker image. Let's build the image and create a container out of it.



A video demonstration of the process of building the Dockerfile is provided in the YouTube link here:
<https://bit.ly/MCVP-Docker-Deployment>.

Run the following commands from the same terminal (where we are in the root directory containing the application files):

1. Build the Docker image and tag it as `sdd:latest` :

```
$ docker build -t sdd:latest .
```

After a long list of outputs, we get the following, telling us that the image is built:

```
⇒ [8/9] COPY . /app 0.8s
⇒ [9/9] RUN ls /app 0.6s
⇒ exporting to image 0.7s
⇒ exporting layers 0.6s
⇒ writing image sha256:1b921c0b5f03a6eab450e518e34198/e54b3f1462ec55f5e8 0.0s
⇒ naming to docker.io/library/sdd:latest 0.0s
```

Figure 18.3: Creating a Docker image

We have successfully created a Docker image with the name `sdd:latest` (where `sdd` is the image name and `latest` is a tag that we gave, indicating its version number). Docker maintains a registry in the system from which all these images are accessible. This Docker registry now contains a standalone image with all the code and logic to run the SDD API.

We can always check in the Docker registry by typing out `$ docker image ls` in Command Prompt.

2. Run the built image with `-p 5000:5000`, forwarding port `5000` from inside the image to port `5000` on our local machine. The last argument is the name of the container being created from the image:

```
$ docker run -p 5000:5000 sdd:latest
```

Port forwarding is important. Often, we don't have a say on which ports the cloud exposes. Hence, as a matter of demonstration, even though our

uvicorn model created a 5000 port for the POST operation, we still use Docker's functionality to route external requests from 5000 to 5000, which is where uvicorn is listening.

This should give a prompt with the last few lines, as follows:

```
> make docker-run
docker run -p 5000:5000 sdd:latest
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:5000 (Press CTRL+C to quit)
```

Figure 18.4: Messages during the application startup

3. Now, run a `curl` request from a new terminal and access the API as described in the previous section, but this time, the application is served from Docker instead:

```
curl -X POST "http://127.0.0.1:5000/predict" -H "accept: application/json" -H "Content-Type: multipart/form-data" -F "file=@/tmp/non-defect.png;type=image/png"
```

Figure 18.5: Executing a predict request

Even though we have not moved anything to the cloud so far, wrapping the API in Docker saves us from having to worry about `pip install` or copy-pasting code ever again.

Now that we have learnt about containerizing the application, let's go ahead, ship, and run the Docker container on cloud in the next section.

Shipping and running the Docker container on the cloud

We will rely on AWS for our cloud requirements. We will use two of AWS's free offerings for our purpose:

- **Elastic Container Registry (ECR):** Here, we will store our Docker image.
- **EC2:** Here, we will create a Linux system to run our API Docker image.

In this section, we will focus only on ECR. Here is a high-level overview of the steps we will follow to push the Docker image to the cloud:

1. Configure AWS on the local machine.
2. Create a Docker repository on AWS ECR and push the `sdd:latest` image.
3. Create an EC2 instance.
4. Install dependencies on the EC2 instance.
5. Create and run the pushed Docker image in *step 2*, on the EC2 instance.



The code in the following sections is also summarized as a video walkthrough here: <https://tinyurl.com/MCVP->

[FastAPI2AWS.](#)

Let's implement the preceding steps, starting with configuring AWS in the next section.

Configuring AWS

We are going to log in to AWS from Command Prompt and push our Docker image. Let's do it step by step:

1. Create an AWS account at <https://aws.amazon.com/> and log in.
2. Install the AWS CLI on your local machine (which contains the Docker image).
3. Verify that it is installed by running `aws --version` in your local terminal.
4. Run `aws configure` in the terminal and give the appropriate credentials when asked. These credentials can be found in IAM section:

```
$ aws configure
AWS Access Key ID [None]: AKIAIOSFODNN7EXAMPLE
AWS Secret Access Key [None]: wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
Default region name [None]: region
Default output format [None]: json
```

We have now logged in to Amazon's services from our computer. In the next section, let's connect to ECR and push the Docker image.

Creating a Docker repository on AWS ECR and pushing the image

Now, we will create the Docker repository, as follows:

1. After configuring, log in to AWS ECR using the following command (the following code is all on one line), where you will need to provide the `region` and your `aws_account_id`, as shown in bold in the following code (remember to use your own values of these variables in every step now onwards):

```
$ aws ecr get-login-password --region regionaws_account_id.region | docker login --username AW'
```

The preceding line of code creates and connects you to your own Docker registry in the Amazon cloud.

2. Create a repository from the CLI by running the following command:

```
$ aws ecr create-repository --repository-name sdd_app
```

The preceding code creates a location in the cloud that can hold your Docker images.

Tag your local image by running the following command so that when you push the image, it will be pushed to the tagged repository:

```
$ docker tag sdd:latest aws_account_idregion.dkr.ecr..amazonaws.com/sdd_app
```

3. Run the following command to push the local Docker image to the AWS repository in the cloud:

```
$ docker push aws_account_idregion.dkr.ecr..amazonaws.com/sdd_app
```

We have successfully created a location in the cloud for our API and pushed the Docker image to this location. As you are now aware, this image already has all the components to run the API. The only remaining aspect is to create a Docker container out of it in the cloud, and we will have successfully moved our application to production!

4. Now, let's go ahead and create a Amazon Linux 2 AMI - `t2.micro` instance with 20 GB of space. When creating the instance, add `Allow http traffic rule` in the `Configure Security Group` section so that the application we will be deploying can be accessed from anywhere.
5. Copy the EC2 instance name that looks like this:

```
ec2-18-221-11-226.us-east-2.compute.amazonaws.com
```

6. Log in to the EC2 instance by using the following command in your local terminal:

```
$ ssh ec2-user@ec2-18-221-11-226.us-east-2.compute.amazonaws.com
```

Next, let's install the dependencies for running the Docker image on the EC2 machine, and then we'll be ready to run the API.

Pulling the image and building the Docker container

The following commands all need to be run in the EC2 console that we logged in to in the previous section:

1. Install and configure the Docker image on a Linux machine:

```
$ sudo yum install -y docker
$ sudo groupadd docker
$ sudo gpasswd -a ${USER} docker
$ sudo service docker restart
```

`groupadd` and `gpasswd` ensure that Docker has all the permissions required to run.

2. Configure AWS in an EC2 instance, as you did earlier, and reboot the machine:

```
$ aws configure
AWS Access Key ID [None]: AKIAIOSFODNN7EXAMPLE
AWS Secret Access Key [None]: wJalrXUtnFEMI/K7MDENG/bPxrFcIYEXAMPLEKEY
Default region name [None]: us-west-2
Default output format [None]: json
$ reboot
```

3. Log in again to the instance from the local terminal using the following command:

```
$ ssh ec2-user@ec2-18-221-11-226.us-east-2.compute.amazonaws.com
```

4. Now, from the EC2 logged-in console (which has Docker installed), log in to AWS ECR (change the region as shown in bold in the following command):

```
$ aws ecr get-login-password --region regionaws_account_id.region | docker login --username AW
```

5. Pull the Docker image from AWS ECR:

```
$ docker pull aws_account_idregion.dkr.ecr..amazonaws.com/sdd_app:latest
```

Finally, run the pulled Docker image in the EC2 machine:

```
docker run -p 5000:5000 aws_account_idregion.dkr.ecr..amazonaws.com/sdd_app
```

We have our API running on EC2. All we have to do is get the public IP address for the machine and run the `curl` request with this address in place of `127.0.0.1`.

6. You can now call a `POST` request from any computer, and the EC2 instance will respond to it, giving us predictions for what type of clothing image we have uploaded:

```
$ curl -X POST "http://54.229.16.169:5000/predict" -H "accept: application/ json" -H "Content-
```

The preceding code results in the following output:

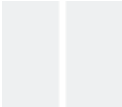
```
{"class": "non-defect", "confidence": "0.6488"}
```

In this section, we were able to install the dependencies for EC2, pull the Docker image, and run the Docker container, to enable any user with the URL to make predictions on a new image.



The whole process is summarized as a video walkthrough here: <https://tinyurl.com/MCVP-FastAPI2AWS>.

Step wise instructions with screenshots are available as a PDF file within the `Chapter18` folder of the associated



GitHub repository as `Shipping and running the Docker container on cloud`.

Now that we understand how to deploy a model, in the next section, we will learn about ways to identify scenarios where the input data (in the real world) is out of distribution when compared to the data used during training.

Identifying data drift

In a typical tabular dataset, it is relatively easy to understand if the incoming data point is an outlier by looking at the summary statistics of the dataset on which the model is trained. However, computer vision models are not as straightforward – we have already seen the quirks that they have in *Chapter 4*, where, just by translating pixels by a few pixels, the predicted class changed. However, this is not the only scenario of data drift in the case of images. There are any number of ways in which the data coming into the production model is different from the data the model was trained on. These may be things that are obvious, such as the image lighting being off and the expected subject in the image being wrong, or subtle things that a human eye cannot see.

In this section, we will understand ways of measuring drift between the input images in real time (real-world images for prediction) and the images that were used during the training of the model.

The reasons we are measuring drift are as follows:

- If the input images in the real world are not like the images that were used during training, then we potentially would be predicting incorrectly.
- Furthermore, we want to get feedback after implementing the model as early as possible so that we can retrain/fine-tune the model.

We'll adopt the following strategy to identify if a new image is out of distribution:

1. Extract the embedding from an intermediate layer (for example, an `avgpool` layer).
2. Perform the above step for both training images as well as any new image from the real world.
3. Compare the embeddings of the new image with the embeddings of all the training images.
4. You can also experiment with the embeddings of only those training images that belong to the same class as that of the new image.
5. If the distance between the embeddings of the new image and the embeddings of training images is high, then the new image is out of distribution.

We'll implement the above in code as follows:



The following code is available as `measuring_drift.ipynb` in the `Chapter18` folder of the book's GitHub repository at


<https://bit.ly/mcnp-2e>

1. Fetch the repo that contains the code to train the model:

```
%%capture
try:
    from torch_snippets import *
except:
    %pip install torch-snippets gitPython lovely-tensors
    from torch_snippets import *
from git import Repo
repository_url = 'https://github.com/sizhky/quantization'
destination_directory = '/content/quantization'
if exists(destination_directory):
    repo = Repo(destination_directory)
else:
    repo = Repo.clone_from(repository_url, destination_directory)
%cd {destination_directory}
%pip install -qq -r requirements.txt
```

2. Train the model:

```
# Change to `Debug=false` in the line below
# to train on a larger dataset
%env DEBUG=true
!make train
```

3. Fetch the training and validation datasets and the dataloaders:

```
from torch_snippets import *
from src.defect_classification.train import get_datasets, get_dataloaders
trn_ds, val_ds = get_datasets(DEBUG=True)
trn_dl, val_dl = get_dataloaders(trn_ds, val_ds)
```

4. Load the model:

```
model = torch.load('model.pth').cuda().eval()
```

5. Get the embeddings of the images used during training:

```
results = []
for ix, batch in enumerate(iter(trn_dl)):
    inter = model.avgpool(model.features(batch[0].cuda()))[:, :, 0, 0].\
        detach().cpu().numpy()

    results.append(inter)
results = np.array(results)
results = results.reshape(-1, 512)
```

6. Get the embeddings of a sample validation image and calculate the distance with the embeddings of the training images:

```
im = val_ds[0]['image'][None].cuda()
tmp = np.array(model.avgpool(model.features(im))[0, :, 0, 0].detach().cpu().\
```

```
numpy())
dists1 = np.sum(np.abs(results - tmp), axis=1)
```

7. Now, let's perform the same exercise but with a completely unrelated image:

```
!wget https://as2.ftcdn.net/v2/jpg/01/42/16/37/1000_F_142163797_YxZaY95j5ckLgb6KoM5KC11Eh9QiZs'
im = cv2.imread(path)[:,:,:-1])
im = cv2.resize(im, (224,224))
im = torch.tensor(im/255)
im = im.permute(2,0,1).float().to(device)
im = im.view(1,3,224,224)
tmp = np.array(model.avgpool(model.features(im))[0,:,0,0]).detach().cpu().\
numpy())
dists2 = np.sum(np.abs(results - tmp), axis=1)
```

8. Let's plot the distribution of the distance of the training images with the validation image and the unrelated (random) image:

```
import seaborn as sns
df = pd.DataFrame(
    {'distance': np.r_[dists1, dists2],
     'source': ['val image']*len(dists1)+['random image']*len(dists2)})
# Just switch x and y
sns.boxplot(y=df["source"], x=df["distance"])
```

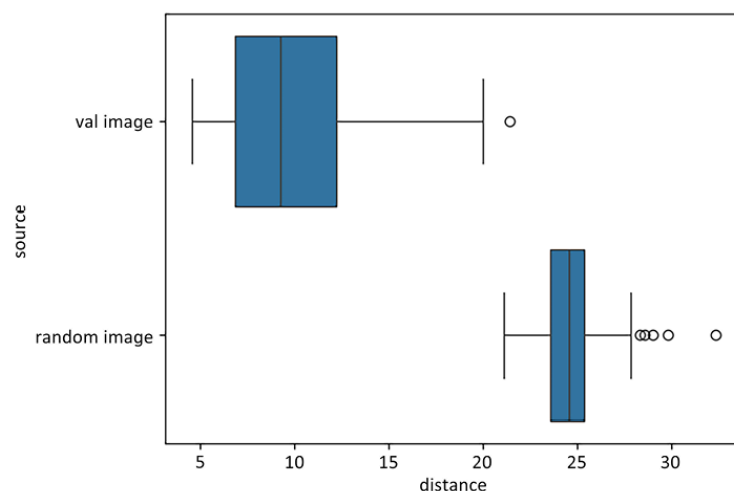


Figure 18.6: Distance distribution between the training images and the validation and random images

From the above, we can see that the distance range of the random image is extremely different when compared to the distance of an image taken from within the distribution. This way, we can understand whether the image that we are predicting on falls within the distribution of training images. Furthermore, when the number (or percentage) of real-world images that are out of distribution falls beyond a threshold, it is worthwhile to consider retraining the model.

In this section, we have calculated the distance between the embeddings of the predicted image and all the images used for training. In this case, it

was easier, given that the number of training images is limited (a few hundred).

What if the number of training images is ~one million? How do we perform the distance calculation relatively quickly? In the next section, we will learn about how vector stores help to solve this problem.

Using vector stores

The intuition of vector stores is as follows: if we can group all the vectors into a certain number of clusters, for a new vector, we can first identify the cluster that it is likely to belong to, and then we can calculate the distance of the new vector with the images that belong to the same cluster.

This process helps to avoid computation across all images, thereby reducing the computation time considerably.

FAISS is an open-source library built by Meta to perform fast approximate similarity search between vectors. There is a wide range of both open-source and proprietary vector store libraries. We strongly recommend you review those once you understand the need for vector stores through the following scenario.

Now that we have an understanding of vector stores, let's go ahead and perform the following steps:

1. Store the training image embeddings in a vector store.
2. Compare the time it takes to retrieve the three closest images to query (validation/real-world) image in the scenarios where:
 1. There is no usage of a vector store
 2. We use a vector store
3. Increase the number of training images, and then compare the time it takes to retrieve the closest images to a given image in the preceding two scenarios.

Let's understand how we can code this:



The following code is available as `vector_stores.ipynb` in the `Chapter18` folder in the GitHub repository at <https://bit.ly/mcvc-2e>.

1. Install the FAISS library:

```
!pip install faiss-gpu
```

The first five steps from the preceding section (on data drift) remain the same – i.e., fetching the training and validation datasets, training and loading the model, and fetching the embeddings of the training images.

2. Index all the vectors of the training images:

```
import faiss
import numpy as np
```

```
index = faiss.IndexFlatL2(results.shape[1]) # L2 distance
index.add(results)
faiss.write_index(index, "index_file.index")
```

In the preceding code, we fetch the training embeddings (results) and index them using the `IndexFlatL2` method. Next, we write the index to disk.

3. Fetch the vector of the new image:

```
im = val_ds[0]['image'][None].cuda()
tmp = np.array(model.avgpool(model.features(im))[0, :, 0, 0].detach().cpu().\
               numpy())
query_vector = tmp.reshape(1, 512).astype('float32')
```

4. Find the most similar vectors:

```
%%time
k = 3 # Number of nearest neighbors to retrieve
D, I = index.search(query_vector.astype('float32'), k)
```

In the preceding code, `D` represents the distance and `I` represents the index of the image that is most similar to the query image. We can see that it took 0.2 ms to search across all images.

5. Increase the number of training image vectors considerably:

```
vectors = np.array(results.tolist()*10000, dtype=np.float32)
print(vectors.shape)
index = faiss.IndexFlatL2(vectors.shape[1]) # L2 distance
index.add(vectors)
faiss.write_index(index, "index_file_960k.index")
```

In the preceding code, we artificially increase the number of training image embeddings by repeating the list of training images 10,000 times. Next, we index the embeddings and write to disk.

6. Calculate the time it takes to search the closest vector to the query vector:

```
%%time
k = 3 # Number of nearest neighbors to retrieve
D, I = index.search(query_vector.astype('float32'), k)
```

Note that, in the preceding scenario, it takes ~ 700 ms to fetch similar vectors.

7. Calculate the time it takes to fetch the three closest vectors without indexing:

```
%%time
distances = np.sum(np.square(query_vector - vectors), axis=1)
```

```
sorted_distances = np.sort(distances)
```

Note that, without indexing, it took ~5 seconds to fetch the closest vectors.

In the following plot, we'll see the time it takes to find the closest matching training images to a given image as the number of training images increases for both scenarios (indexing and non-indexing):

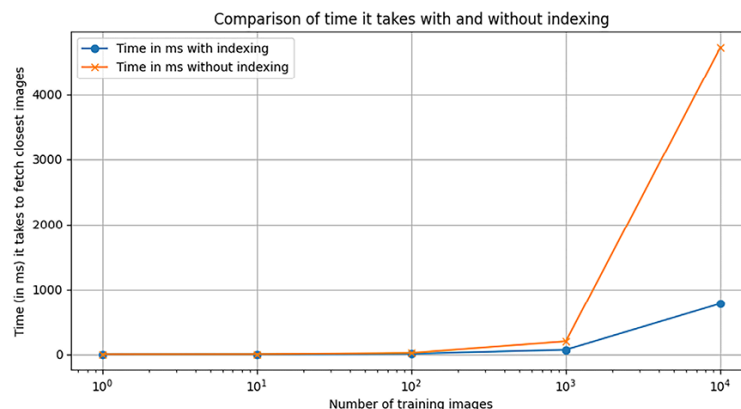


Figure 18.7: Comparison of the time it takes to identify the closest matching images with and without indexing and over an increasing number of training images

Note that the difference between having an index versus no index grows exponentially with an increasing number of training images.

Nonetheless, in a world where information retrieval is paramount for businesses, getting the solution in the fastest time is important, and vector stores are a crucial way to save time.

Summary

In this chapter, we learned what additional steps are required in moving a model to production. We learned what an API is and what its components are. After creating an API with the use of FastAPI, we glanced at the core steps of creating a Docker image of the API, creating a Docker container, and pushing the Docker container to cloud so that predictions can be made from any device. Then, we learned about ways to identify images that are out of distribution from the original dataset. Additionally, we also learned about leveraging FAISS to calculate the distance with similar vectors much faster.

To summarize, we have seen all the individual steps to deploy a model to production, including building a Docker container, deploying on the AWS cloud, identifying data drift and thereby understanding the scenario of when to re-train the model, and performing image similarity much faster so that we can identify data drift with less compute.

Images are fascinating. Storing them has been one of humanity's earliest endeavors and is one of the most powerful ways to capture content. The ease of capturing images in the 21st century has opened up multitudes of problems that can be solved with or without human intervention. In this

book, we have covered some of the most common as well as some more modern tasks using PyTorch – image classification, object detection, image segmentation, image generation, manipulating a generated image, leveraging foundation models for various tasks, combining computer vision with NLP techniques, and reinforcement learning. We covered the working details of various algorithms from scratch. We also learned how to formulate a problem, capture data, train models, and infer from the trained models. We understood how to pick up code bases/pretrained models and customize them for our tasks, and finally, we learned about deploying our model in an optimized manner and incorporating drift monitoring.

We hope that you have picked up the necessary skills to handle images like it's second nature and solve tasks that interest you.

Most importantly, we hope that this has been a joyful journey for you and that you have enjoyed reading the book as much as we have enjoyed writing it!

Questions

1. What is the REST API and what does it do?
2. What is Docker and why is it important for deploying deep learning applications?
3. What is a simple and common technique for detecting unusual or novel images in a production setting that differ substantially from those used during training?
4. How can we speed up the similarity search of an image with a large volume of vectors (millions)?

Learn more on Discord

Join our community's Discord space for discussions with the authors and other readers:

<https://packt.link/modcv>

